Chat with Your PDF: A RAG-Based Document Question-Answering System

Project Report | Prepared by K G SHRUJITHA

1. Abstract

Chat with Your PDF is an AI-driven web application that lets users get answers from a PDF by simply asking questions in plain language. The user uploads a file, the system reads and splits the text into overlapping segments, converts each segment into a vector embedding, and stores the vectors in a FAISS index. When a question is asked, the most semantically similar segments are fetched and passed as context to a large language model hosted on Groq, which composes the final answer. The whole experience is delivered through a lightweight Streamlit interface. By joining document parsing, semantic search, and text generation, the project makes information lookup in long documents quick and effortless.

2. Introduction

Study notes, research papers, manuals, and official reports are mostly shared as PDFs. Finding one specific detail in such files often means scrolling through many pages, and ordinary keyword search fails when the question uses different words from the document. This project solves the problem with Retrieval-Augmented Generation (RAG): the system first retrieves the passages most related to the question and then lets a language model write the answer using only that material.

3. Problem Statement

Readers waste significant time hunting for specific facts in lengthy PDFs. The goal is to build an easy-to-use tool where a user can upload a PDF, ask questions about it, and receive relevant, context-based answers without reading the whole file.

4. Objectives

- Provide a simple browser-based interface for uploading PDF files.
- Extract text from the document and divide it into overlapping chunks.
- Generate embeddings for each chunk and index them with FAISS for fast similarity search.
- Pass the retrieved chunks to a Groq-hosted language model to produce grounded answers.
- Support multiple follow-up questions on the same processed document.

5. System Architecture and Workflow

The application works as a pipeline. After a PDF is uploaded, its pages are read and the text is extracted. The text is cut into small overlapping chunks so that each passage can be searched on its own. A Sentence Transformers model turns every chunk into an embedding, and these vectors are saved in a FAISS index. When the user types a question, it is converted into an embedding using the same model and compared with the stored vectors. The top-matching chunks are inserted into a prompt, which is sent to the language model through the Groq API. The reply is then shown on the Streamlit page.

- **Upload:** The user selects a PDF containing selectable text.

- **Extraction:** PyPDF reads the text from each page.
- **Chunking:** The text is divided into overlapping segments.
- **Embedding and Indexing:** Sentence Transformers creates vectors; FAISS stores them for search.
- **Retrieval:** The nearest chunks to the question are selected.
- **Generation:** The chunks are added to the prompt and the Groq model writes the answer.
- **Display:** Streamlit presents the answer to the user.

6. Technologies Used

- **Python:** The main language that ties all modules together.
- **Streamlit:** Creates the web interface for uploading files and asking questions.
- **PyPDF:** Pulls selectable text out of PDF pages.
- **Sentence Transformers:** Produces embeddings used for semantic matching.
- **FAISS:** Performs fast vector similarity search.
- **Groq API:** Gives access to a hosted language model for answer generation.
- **python-dotenv:** Keeps the API key in a private .env file.

7. Project Structure

- **app.py:** The Streamlit front end that links user actions to the backend.
- **document_loader.py:** Reads and cleans text from uploaded PDFs.
- **vector_store.py:** Builds embeddings and manages the FAISS index.
- **rag_pipeline.py:** Controls retrieval and the call to the language model.
- **prompt.py:** Holds the instruction template that guides the model's answers.
- **requirements.txt:** Lists all required Python libraries.
- **tests/:** Unit tests for PDF loading, chunking, and retrieval.

8. Implementation and Execution

The project runs as a Python application. After installing the packages from requirements.txt and storing the Groq API key in a local .env file, the app is launched with `streamlit run app.py`. In the browser, the user uploads a PDF and clicks the process button. When indexing completes, a question can be typed and submitted, and the answer appears on screen. The API key must stay confidential and should never appear in screenshots, screen recordings, or the submitted report.

9. Testing and Expected Results

Separate tests cover PDF loading, chunking, and retrieval. Manual checks include uploading a text-based PDF, confirming that text is extracted, asking a question whose answer is known, and comparing the response with the source document. A follow-up question confirms that the processed document can be queried repeatedly. Edge cases such as empty files, scanned PDFs without selectable text, invalid uploads, a missing API key, and questions unrelated to the document are also checked. Actual results vary with the PDF, retrieval quality, and the language model's output.

10. Advantages

- Saves time compared with reading long documents manually.
- Accepts questions in natural language rather than exact keywords.
- Grounds answers in passages from the uploaded PDF.
- Uses a clean, modular code structure that is easy to extend.
- Works with notes, reports, manuals, and other text-based PDFs.

11. Limitations and Future Scope

The system needs selectable text, so scanned or image-only PDFs would require Optical Character Recognition (OCR). Retrieval can miss relevant passages, and the language model may still give incomplete or wrong answers. The Groq API also needs a valid key and an internet connection. Future work includes showing page numbers as sources, adding OCR support, handling out-of-scope questions more gracefully, measuring retrieval quality with a labelled test set, and allowing several documents to be searched together.

12. Conclusion

Chat with Your PDF shows how Retrieval-Augmented Generation can turn static documents into an interactive knowledge source. It combines text extraction, chunking, embeddings, FAISS search, and language-model generation in a simple Streamlit application. Although answer accuracy depends on the document and the retrieval step, the project offers a practical base for building more reliable and user-friendly document assistants.